How Do SVM Kernels Change Classification?

Tutorial objectives

- Explain why SVMs are useful on small and medium tabular datasets,
- Choose between linear, polynomial, and RBF kernels,
- Implement a scaled SVM pipeline in Python,
- Interpret confusion matrices and boundary plots, and
- Tune C and gamma without using the test set as a tuning tool.

Why this technique is worth learning

Support Vector Machines (SVMs) are a good choice for tabular classification tasks, especially when the dataset is fairly small and the pattern in the data may be either simple or more complex. In this tutorial, I focus on one practical question: how does the choice of kernel change what an SVM is able to learn? To explore this, I use the Banknote Authentication dataset and compare three common kernels: linear, polynomial, and radial basis function (RBF). I then take a closer look at the RBF kernel to show how the hyperparameters **C** and **gamma** affect the model's behaviour. The full implementation is included in the accompanying notebook, while this tutorial concentrates on the key ideas, the most useful code examples, and the figures that make the results easier to understand.

SVMs are still popular because they have a simple idea behind them but can still give very strong results. The aim is to separate the classes with a boundary that leaves as much space as possible between them. The points that matter most in deciding where that boundary sits are called support vectors. In practice, this makes SVMs useful for a few reasons. They often work well on small and medium-sized tabular datasets, they can handle both straightforward and more complex patterns depending on the kernel used, and they are interpretable enough to give some insight into why the model makes its decisions.

Dataset briefly

Attribute	Value
Dataset	Banknote Authentication
Instances	1,372
Features	4 numeric features
Target	Binary classification
Missing values	None reported
Task type	Real-world classification task that is still compact enough to tune carefully

How the method works

In a linear SVM, the decision boundary is a hyperplane, so using a linear kernel means the classifier can only separate the classes with a straight boundary. A polynomial kernel makes the model more flexible by allowing it to capture curved relationships, depending on the degree chosen. The RBF

kernel is even more flexible, because it works by measuring similarity locally around data points in feature space. In simple terms, the kernel decides what kind of shape the SVM can learn.

This matters in practice. If the classes are close to being linearly separable, a linear kernel can work well and is usually easier to tune. If the data have a more complex structure, such as curved patterns, clusters, or overlapping regions, the RBF kernel is often a better option. Polynomial kernels can also model non-linear patterns, but they are generally more sensitive to parameter choices and can be harder to tune well.

Kernel	Boundary shape	When it helps	Main risk
Linear	One global straight boundary	Strong baseline for nearly linear tabular data	Underfitting curved structure
Polynomial	Curved boundary of chosen degree	Useful when interactions have structured curvature	Sensitive to degree and coefficient
RBF	Flexible local boundary	Mixed, clustered, or clearly non-linear patterns	Overfitting if C and gamma are too high

Minimal SVM pipeline used in the notebook

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

model = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf", C=10, gamma=0.1))
])
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

What the code and figures demonstrate

Compact loop for comparing kernels

```
kernels = {
    "Linear": SVC(kernel="linear", C=1),
    "Polynomial": SVC(kernel="poly", degree=3, C=1, gamma="scale"),
    "RBF": SVC(kernel="rbf", C=1, gamma="scale"),
}

for name, model in kernels.items():
    pipe = Pipeline([("scaler", StandardScaler()), ("svc", model)])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    print(name, accuracy_score(y_test, y_pred))
```

The code is used to train three kernels on the same four-feature dataset, generate evaluation plots, and then create a controlled RBF tuning experiment. Each figure answers a different practical question about when the technique is helpful and how someone could use it in their own work.

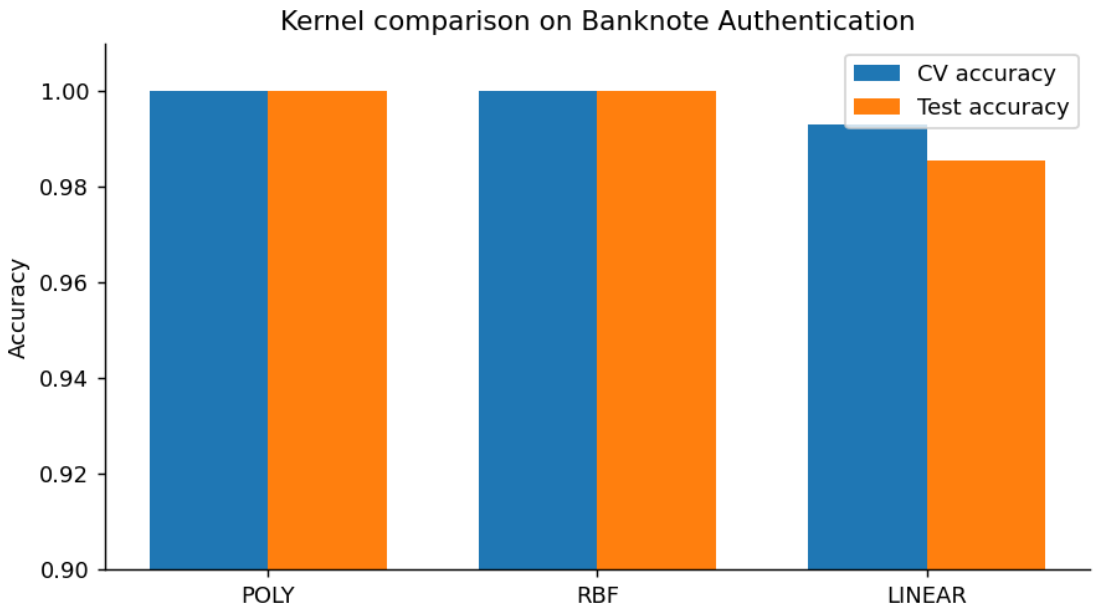

Figure 1: Linear, polynomial, and RBF SVM accuracy on the Banknote dataset.

Figure 1 shows the cross-validation and test accuracy of the three kernels on the full four-feature dataset. All three models perform very well, but there are small differences. The linear SVM reaches about 98.5% test accuracy, the polynomial SVM about 99.7%, and the RBF SVM 100% on this split. This suggests two main points: the dataset is highly suitable for SVM classification, and allowing a more flexible non-linear boundary helps reduce the small number of errors made by the linear model.

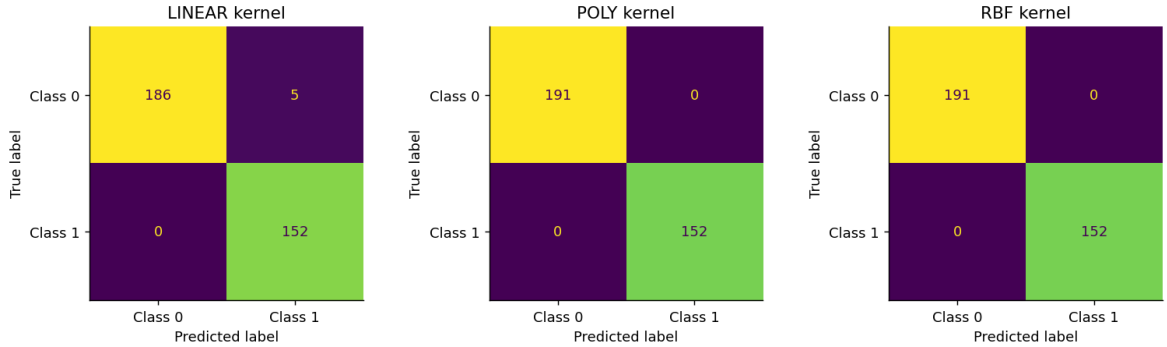

Figure 2. Confusion matrices show that polynomial and RBF reduce the linear model's errors.

Accuracy on its own does not show where the mistakes happen, so Figure 2 includes confusion matrices for a clearer view. The linear kernel makes five errors, all in the same direction: a small number of class 0 samples are predicted as class 1, while class 1 is classified correctly throughout. This is important in practice, because the cost of a false positive may not be the same as the cost of a false negative. The polynomial and RBF models reduce these errors considerably, and on this split the RBF model classifies every test sample correctly.

Checking where the mistakes happen

```
from sklearn.metrics import ConfusionMatrixDisplay

best_model = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(kernel="rbf", C=1, gamma="scale"))
])

best_model.fit(X_train, y_train)
ConfusionMatrixDisplay.from_estimator(best_model, X_test, y_test)
```

A confusion matrix shows not only how many errors occur, but also which class is being confused with which.

Visualizing the decision boundary

Figure 3 helps show why the kernels behave differently. To make the decision boundary easier to see, the dataset was reduced to two dimensions using PCA, although the models themselves were still trained and tested on all four original features. In this PCA view, the linear kernel produces a single straight diagonal boundary. The polynomial kernel bends that boundary and follows the central region more closely, while the RBF kernel is the most flexible and adjusts more locally around the mixed area. The black circles mark the support vectors, which are the points closest to the margin. These points are especially important because they have the strongest influence on where the boundary is placed.

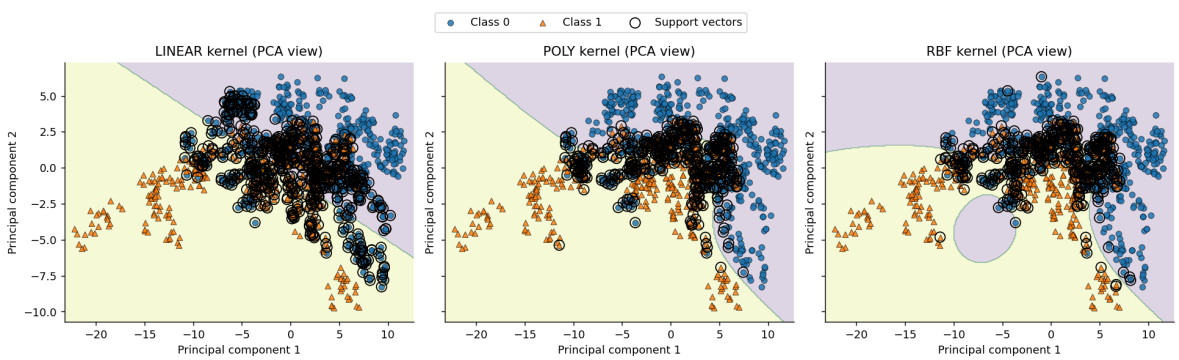

Figure 3. **PCA is used only for visualization; evaluation uses all four features. Support vectors are circled in black.**

This figure is especially valuable for teaching because it translates a kernel choice into a visible modelling decision

Short tuning pattern for an RBF SVM

```
param_grid = {
    "svc__C": [0.1, 1, 10, 100],
    "svc__gamma": [0.01, 0.1, 1, "scale"]
}
search = GridSearchCV(
    Pipeline([("scaler", StandardScaler()),
              ("svc", SVC(kernel="rbf"))]),
```

```
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)
```

This second code block shows how a reader could move from a working baseline to a reproducible hyperparameter search on their own data.

Deep dive: why C and gamma matter

Figure 4 shows why RBF SVMs are powerful but also why they need tuning. The parameter C controls how heavily training mistakes are penalised. A small C allows a softer margin; a large C pushes the model to fit the training set more tightly. The parameter gamma controls how local each support vector's influence becomes. Low gamma gives a smoother, more global boundary. High gamma makes the model very local and can create small islands in feature space.

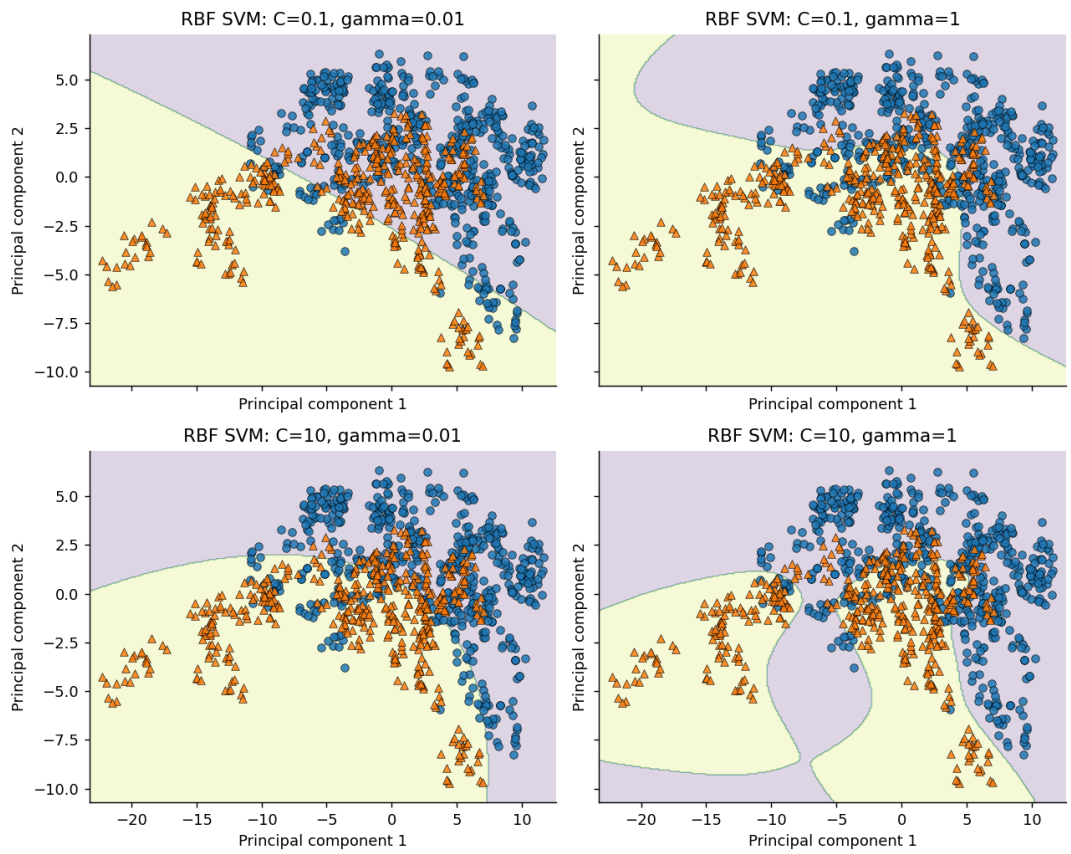

Figure 4. RBF behaviour under different values of C and gamma in PCA space.

In the figure, the bottom-right panel (high C, high gamma) is the clearest warning sign: the boundary becomes much more intricate, which is exactly how overfitting starts. In contrast, moderate values such as C = 10 and gamma = 0.1 achieved perfect cross-validation and test accuracy in the accompanying notebook.

How this can improve work

Step	What to do in practice
1	Start with a scaled linear SVM to get a clear baseline.

2	If mistakes remain and the pattern looks non-linear, try an RBF SVM.
3	Tune C and gamma with cross-validation, not on the test set.
4	Inspect a confusion matrix to understand which mistakes matter.
5	Use a reduced-dimension plot to explain or debug the boundary.

In practice, this technique can be used in a straightforward way. A good starting point is a scaled linear SVM as a baseline model. If the results suggest that the classes are not separated well by a straight boundary, it makes sense to move to an RBF SVM. The parameters **C** and **gamma** should then be tuned with cross-validation, rather than adjusted by repeatedly checking the test set. After that, it is useful to look at the confusion matrix as well as the overall accuracy, and to use a low-dimensional visualization to see where the model is finding the problem difficult. This makes kernel choice more systematic, because it is guided by the structure of the data instead of simple guesswork.

One final practical note is scale: classic kernel SVMs are best for small and medium datasets. On very large datasets, fit time can become impractical, so linear approximations or other models may be better.

Conclusion

This tutorial shows that SVMs are not only strong classifiers, but also a useful way to understand how model choice affects results. One of the clearest things about SVMs is that changing the kernel changes the shape of the decision boundary, so the effect can be seen quite directly. On the Banknote dataset, all three kernels perform well, but the non-linear models reduce the small number of errors left by the linear baseline, with the tuned RBF model giving the best result. From a practical point of view, the main lesson is that kernel choice should be treated as part of the modelling process: a linear kernel is a good starting point for simplicity, a polynomial kernel can help with more structured curvature, and an RBF kernel is often the best option when the pattern is clearly non-linear and careful tuning is possible.

Reference list

1. van Engelen, J.E. and Hoos, H.H. (2020) 'A survey on semi-supervised learning', *Machine Learning*, 109, pp. 373–440.
2. Triguero, I., García, S. and Herrera, F. (2015) 'Self-labeled techniques for semi-supervised learning: taxonomy, software and empirical study', *Knowledge and Information Systems*, 42(2), pp. 245–284.
3. scikit-learn developers (n.d.) *SelfTrainingClassifier*. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.semi_supervised.SelfTrainingClassifier.html
4. scikit-learn developers (n.d.) *Semi-supervised learning*. Available at: https://scikit-learn.org/stable/modules/semi_supervised.html

GitHub Repository: (**https://github.com/rt25aaz/Machine-Learning-Tutorial-by-Talha.git)** https://github.com/rt25aaz/Machine-Learning-Tutorial-by-Talha.git